

Beyond Technologies Full Stack AI Engineer - What to Learn

MUST – learn before interviews

Durable execution: checkpoints, retries, idempotency

What it is: making a multi-step AI/agent workflow survive crashes and retries without corrupting state or double-running side effects (e.g. sending an email twice). A checkpoint saves progress so a retry resumes instead of restarting; idempotency means running the same step twice produces the same result as running it once.

Why the JD wants it: the role explicitly calls for “durable state, checkpoints, retries, timeouts, idempotency, recovery and human approval.” This is the single most JD-specific concept outside your current experience.

Bridge from your background: Vibe’s Inngest AgentKit loop already gives you durable background jobs and retries – Inngest is literally a durable-execution framework. Read Inngest’s docs on “durable functions” and “steps” – you already use the pattern, you just need the vocabulary and the deeper failure-mode reasoning (what happens if step 3 of 5 fails and retries – did step 2’s side effect already happen?).

Document AI & OCR

What it is: extracting structured text/data from scanned documents, PDFs, and images (e.g. invoices, forms) using OCR (optical character recognition) plus layout-aware models that understand tables and fields, not just raw text.

Why the JD wants it: “document processing” and “Document AI and OCR” are named directly as a core workflow this team builds.

Bridge from your background: Skimly already does PDF ingestion (via a PDFLoader) and you’ve handled file uploads with UploadThing/Multer across projects. The delta is OCR-specific tooling – look at AWS Textract, Google Document AI, or open-source Tesseract/unstructured.io, and understand the extract -> structure -> validate pipeline shape.

Model Context Protocol (MCP) – practical depth

What it is: an open protocol (from Anthropic) standardizing how an AI agent connects to external tools/data sources – think “USB-C for AI tools.” An MCP server exposes tools/resources; an MCP client (like Claude Code, which you already use daily) calls them.

Why the JD wants it: explicitly named as a “useful additional experience” – agent protocols such as MCP.

Bridge from your background: you already use an MCP client daily (Claude Code). The gap is building an MCP *server* – read the MCP spec’s server quickstart and try wrapping one of your own APIs (e.g. Echo’s search tool) as an MCP tool.

GOOD – strengthen if time allows

AI evaluations & guardrails

What it is: systematic testing of LLM outputs (not just unit tests) – eval sets with expected behavior, automated scoring/rubrics, and guardrails (input/output filters, schema validation, refusal handling) to keep an agent safe and on-task in production.

Why the JD wants it: named directly as part of “owning quality in production.”

Bridge from your background: your Zod-typed tool schemas in Echo/Vibe are already a lightweight guardrail (structured output validation). Look at frameworks like Braintrust, promptfoo, or writing your own eval harness – a set of input/expected-output pairs run against your agent on every change.

Distributed systems & queues (deeper theory)

What it is: durable message queues (Kafka, RabbitMQ, SQS) for decoupling services, at-least-once vs. exactly-once delivery semantics, and coordinating work across multiple services reliably.

Why the JD wants it: named under “backend API design ... databases, queues and distributed systems.”

Bridge from your background: Inngest’s background jobs are a managed analog of a queue – you understand the queueing *pattern* already. Read a Kafka or SQS fundamentals guide focused on delivery guarantees and consumer groups, since that’s the interview-testable gap.

Real-time voice systems (deeper)

What it is: streaming audio pipelines, turn-taking/interruption handling, and latency budgets for a live voice agent conversation (distinct from a simple STT-then-LLM-then-TTS pipeline).

Why the JD wants it: named as “useful additional experience: real-time voice systems.”

Bridge from your background: you already shipped Vapi-based voice agents in Echo – read Vapi’s docs on interruption handling and turn detection to go one level deeper than the integration you’ve already built.